

Facultad de Ingeniería

Programa de Ingeniería de Software

Informe de Simulación

Termómetro Digital con PIC16F887

Presentado por

Dylan Nuñez
Andrés Vega
Giovanny Poveda

Presentado a

Ing. Diego Restrepo

Video del proyecto

https://youtu.be/WXi4rC4VkVw

3 de noviembre de 2026

1. Resumen

Se abrió el proyecto del termómetro digital en Proteus 8 Professional y se ejecutó la simulación con el firmware compilado cargado en el microcontrolador PIC16F887.

La simulación digital arranca y corre correctamente: la pantalla LCD se actualiza, la tarjeta SD virtual responde "OK" y el escaneo I2C detecta el LCD. El sensor DHT11 presenta un error de lectura persistente ("E1"). Al comparar el firmware con el esquemático se encontró la causa: el sensor está conectado a un pin distinto del que lee el programa (sección 5).

2. Configuración del proyecto

Elemento	Detalle
Proyecto Proteus	`termometro_analitico_16f887.pdsprj`
Firmware cargado	`prueba.X.production.hex`
Microcontrolador	U1 — PIC16F887 (DIL40)
Frecuencias de reloj probadas	20 MHz (la del firmware), 4 MHz y 8 MHz
Periféricos	LCD1 (LM016L por I2C vía PCF8574A, U2), DHT11 (U3), SD1 (tarjeta SD/MMC) y botones de entrada. El firmware también usa un RTC DS1307, que no está en el esquemático
Pin DATA del DHT11	Esquemático: RC5/SDO (pin 24), con pull-up R4 = 4.7 kΩ. Firmware: RC2 (pin 17)

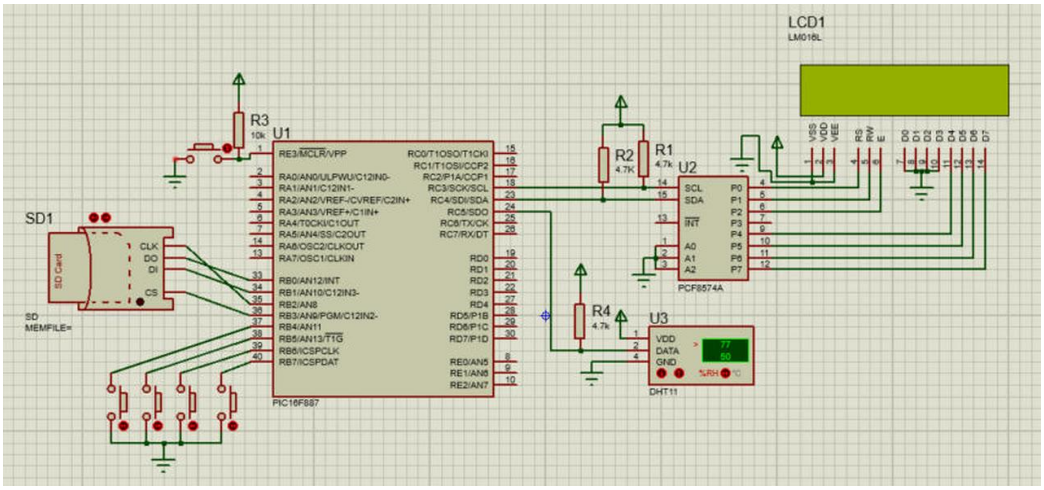


Figura 1. Esquemático del termómetro en Proteus 8.

3. Incidencias resueltas

Incidencia	Causa	Solución aplicada
`Fatal simulation error(s) encountered` al arrancar	La Card Image File de SD1 apuntaba a `MMC_FAT32.mmc` en una ruta inexistente (`..\Downloads\MMC_FAT32`)	Se cambió a `sdcard.mmc` dentro de la carpeta del proyecto, donde Proteus sí puede crear el archivo virtual
El Program File de U1 apuntaba a una ruta inexistente	El proyecto guardaba la ruta de compilación original del autor en MPLAB X (`MPLABX\prueba.X\dist`)	Se asignó manualmente el archivo `.hex` de la carpeta "prueba"

Con ambas correcciones la simulación inició sin errores fatales y se mantuvo corriendo ("ANIMATING").

4. Resultados de la simulación

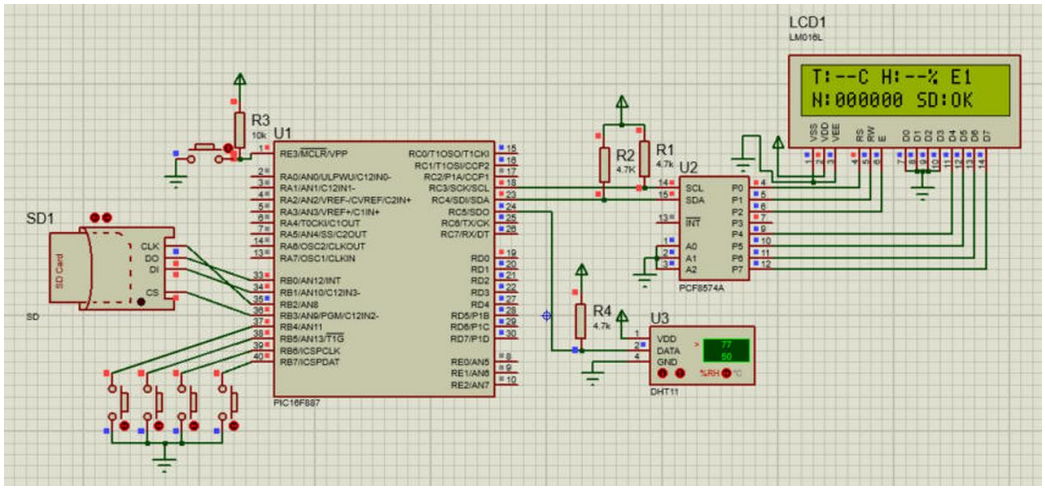


Figura 2. Simulación en ejecución: el LCD se actualiza y la tarjeta SD reporta "OK".

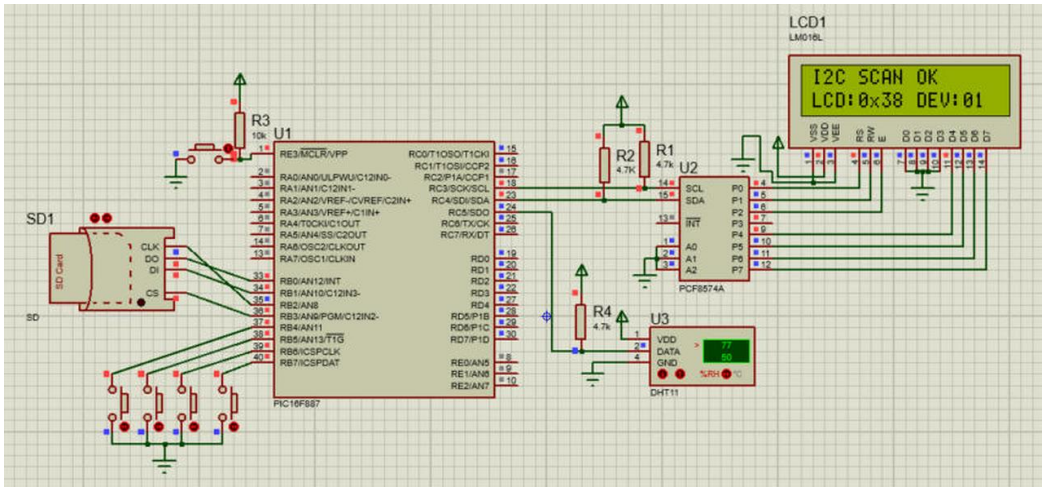


Figura 3. Escaneo I2C exitoso a 4 MHz: el microcontrolador y el bus I2C funcionan.

5. Error "E1" del DHT11

5.1 Síntoma

El LCD muestra `T:--C H:--% E1` de forma persistente. En el firmware, E1 significa que `dht_read()` devolvió 1: el sensor no respondió al inicio de la lectura.

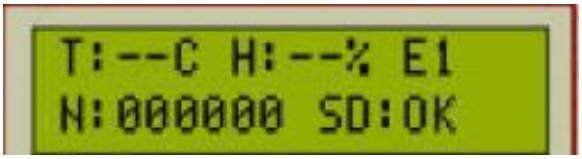


Figura 4. Error E1 mostrado en el LCD.

5.2 Causas descartadas

- **Circuito del sensor:** la línea DATA tiene pull-up R4 = 4.7 kΩ, la configuración estándar correcta.
- **Ejecución del firmware:** con las tres frecuencias probadas el bus I2C funcionó ("I2C SCAN OK") y los textos del LCD coinciden con el código fuente. El firmware está compilado para 20 MHz, así que esa es la frecuencia que debe usarse.

5.3 Causa identificada

El firmware lee el sensor en RC2 (`#define DHT_PIN PORTCbits.RC2`, pin 17), pero en el esquemático DATA está conectado a RC5/SDO (pin 24, Figura 5). Además, RC5 queda configurado como salida en bajo (`TRISC = 0x1C`, `PORTC = 0x00`), lo que mantiene la línea en 0. El PIC nunca recibe la respuesta del sensor y por eso muestra E1.

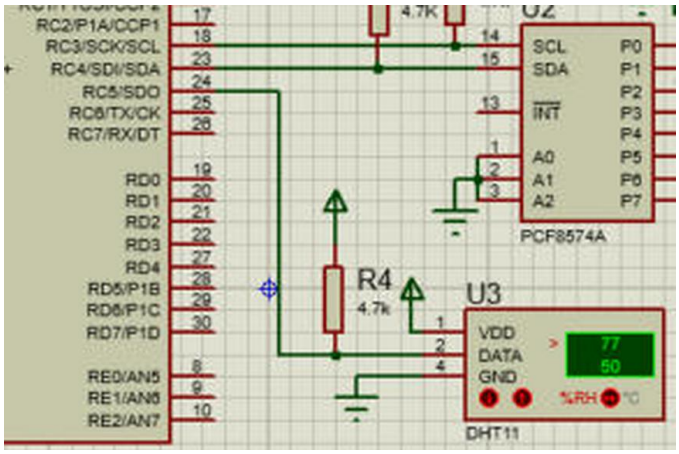


Figura 5. DATA del DHT11 (U3) conectado a RC5/SDO, pin 24; el firmware lo espera en RC2, pin 17.

5.4 Solución propuesta

Mover la conexión DATA del pin 24 (RC5) al pin 17 (RC2) en Proteus y simular a 20 MHz, sin modificar el firmware. Queda pendiente repetir la simulación para confirmarlo. Si después aparece E2 o E3, el siguiente punto por revisar es la temporización: el modelo VSM del DHT11 es sensible a pequeñas diferencias de tiempo en la lectura bit a bit.

5.5 Observación: reloj DS1307

El firmware usa un RTC DS1307 en I2C, pero no está colocado en el esquemático; por eso el escaneo reporta un solo dispositivo (`DEV:01`, Figura 3). Sin él, la vista RELOJ muestra `--`. No causa el error E1.

6. Firmware

6.1 Descripción

Estación termohigrométrica con planificador cooperativo (tick de 20 ms por Timer1). Lee el DHT11 cada 10 s, guarda los registros en la microSD y muestra 13 vistas en el LCD: valores actuales, reloj, muestras, último registro, botones, estadísticas y pronóstico de temperatura y humedad.

6.2 Depuración del código

Se quitaron definiciones sin uso y comentarios repetidos, y los bloques duplicados se unieron en funciones auxiliares (`lcd_bl_apply`, `lcd_dht_status`, `lcd_err`, `lcd_nl`, `lcd_su2`, `sd_block_cmd`, `spi_skip`). Se conservan las 44 funciones originales, las 13 vistas y la temporización; la rutina de lectura del DHT11 no se modificó. El código depurado está en el Anexo A.

Medida	Original	Depurado	Cambio
Líneas de código	1 096	961	−12,3 %
Tamaño del archivo fuente	32 680 bytes	29 612 bytes	−9,4 %
Memoria de programa	5 508 palabras	5 375 palabras	−2,4 %
RAM	202 bytes	204 bytes	+2 bytes

Ambas versiones compilan sin errores con XC8 v4.00. La versión depurada se verificó compilándola; falta probarla en la simulación.

Anexo A. Código fuente depurado

```
/*
 * Estacion termohigrometrica - PIC16F887 a 20 MHz, compilador XC8.
 * Planificador cooperativo con tick de 20 ms (Timer1).
 * RB0=MISO RB1=MOSI RB2=SCK RB3=CS (microSD por SPI software)
 * RB4=anterior RB6=siguiente RB5=muestrear
 * RC2=DHT11 RC3=SCL RC4=SDA RD0=LED de actividad
 * I2C: LCD 16x2 con PCF8574 y RTC DS1307
 * Vistas: 0 actual, 1 reloj, 2 muestras, 3 ultimo registro, 4 botones,
 * 5-7 temperatura, 8-10 humedad, 11-12 pronostico T/H
 */
#include <xc.h>
#include <stdint.h>

#define _XTAL_FREQ 20000000UL

#pragma config FOSC = HS
#pragma config WDTE = OFF
#pragma config PWRTE = ON
#pragma config MCLRE = ON
#pragma config CP = OFF
#pragma config CPD = OFF
#pragma config BOREN = ON
#pragma config IESO = OFF
#pragma config FCMEN = OFF
#pragma config LVP = OFF

/* CONFIGURACION */
#define DHT_PIN PORTCbits.RC2
#define DHT_TRIS TRISCbits.TRISC2
#define TRISC_INIT_MASK 0x1Cu /* RC2 DHT y RC3/RC4 I2C como entradas */

#define LCD_ADDR_PCF8574_MIN 0x20u
#define LCD_ADDR_PCF8574_MAX 0x27u
#define LCD_ADDR_PCF8574A_MIN 0x38u
#define LCD_ADDR_PCF8574A_MAX 0x3Fu
#define LCD_RS 0x01u
#define LCD_EN 0x04u
#define LCD_BL 0x08u

#define SAMPLE_PERIOD_TICKS 500u /* 500 * 20 ms = 10.0 segundos */
#define RTC_PERIOD_TICKS 10u /* 10 * 20 ms = 0.2 segundos (refresco reloj exacto) */
#define BL_TIMEOUT_TICKS 150u /* 150 * 0.2 s = 30.0 segundos luz auto */
#define LCD_RETRY_PERIOD 30u /* 30 * 20 ms = 0.6 segundos */
#define HISTORY_SIZE 24u
#define VIEW_COUNT 13u
#define PAGE_BTN 4u /* vista de diagnostico de botones */
#define LOG_START_SECTOR 200000UL /* LOG_START_SECTOR-1 reservado para indice */
#define LOG_MAX_RECORDS 100000UL
#define SD_RETRY_SAMPLES 5u
#define I2C_TIMEOUT 4000u
#define SD_HALF_US 2u

/* Estados de la tarea no bloqueante de microSD */
#define SD_TASK_IDLE 0u
#define SD_TASK_WRITE_DATA 1u
#define SD_TASK_WRITE_INDEX 2u

/* ESTADO GLOBAL */
static uint8_t portb_shadow = 0x0Au; /* CS alto, MOSI alto, SCK bajo */
static uint8_t lcd_ok = 0u;
static uint8_t lcd_addr = 0u;
static uint8_t i2c_devices_found = 0u;
static uint8_t lcd_backlight = LCD_BL;
static uint8_t bl_timer = 0u;
static uint8_t sd_ready = 0u;
static uint8_t sd_error = 1u;
static uint8_t sd_retry = 0u;
static uint8_t sd_sdhc = 0u;
```

```
static uint8_t sd_task_state = SD_TASK_IDLE;
static __uint24 record_count = 0U;
static uint16_t sess_saved = 0u; /* registros guardados en esta sesion */

/* 0:hour, 1:min, 2:sec, 3:dow, 4:day, 5:month, 6:year */
static uint8_t rtc_time[7];
#define rtc_hour (rtc_time[0])
#define rtc_sec (rtc_time[2])
#define rtc_year (rtc_time[6])

static uint8_t rtc_ok = 0u;
static uint8_t last_sec = 0xFFu;
static uint8_t dht_error = 0u;
static uint8_t temperature = 0u, humidity = 0u;

static uint8_t history[HISTORY_SIZE * 2u]; /* 0..23: hum, 24..47: temp */
static uint8_t history_pos = 0u;
static uint8_t history_count = 0u;
static uint8_t work_buf[32]; /* Compartido entre SD (32 B), DHT y sort */
static uint8_t last_hour = 0xFFu;
static uint16_t soft_hour_ticks = 0u;
static uint8_t fc_prev[2], fc_last[2], fc_count = 0u; /* 0:hum, 1:temp */

static uint8_t page = 0u;
static uint8_t last_page = 0xFFu;
static uint8_t btn_prev = 0u;
static uint8_t btn_cur = 0u; /* estado actual de botones (vista BOTONES) */
static uint16_t tick_sample = SAMPLE_PERIOD_TICKS - 1u;
static uint8_t tick_rtc = RTC_PERIOD_TICKS - 1u;
static uint8_t lcd_retry_ticks = 0u;
static uint8_t needs_redraw = 1u;

/* UTILIDADES */
static void delay_ms(uint8_t ms) {
    while (ms-- != 0u) __delay_ms(1);
}

static void put2(uint8_t pos, uint8_t v) {
    uint8_t tens = '0';
    if (v > 99u) v = 99u;
    while (v >= 10u) { v -= 10u; ++tens; }
    work_buf[pos] = tens;
    work_buf[(uint8_t)(pos + 1u)] = (uint8_t)('0' + v);
}

/* I2C MSSP */
/* Funcion hoja sin sub-llamadas para proteger la pila de 8 niveles del PIC16 */
static uint8_t i2c_cmd(uint8_t mask) {
    uint16_t t = I2C_TIMEOUT;
    while ((SSPCON2 & 0x1Fu) || SSPSTATbits.R_W) { if (--t == 0u) return 0u; }
    PIR2bits.BCLIF = 0;
    SSPCON2 |= mask;
    t = I2C_TIMEOUT;
    while ((SSPCON2 & 0x1Fu) || SSPSTATbits.R_W) { if (--t == 0u) return 0u; }
    return (uint8_t)!PIR2bits.BCLIF;
}

#define i2c_start() i2c_cmd(0x01u)
#define i2c_restart() i2c_cmd(0x02u)
#define i2c_stop() i2c_cmd(0x04u)

static uint8_t i2c_send(uint8_t data) {
    uint16_t t = I2C_TIMEOUT;
    PIR1bits.SSPIF = 0;
    SSPCONbits.WCOL = 0;
    SSPBUF = data;
    if (SSPCONbits.WCOL) return 0u;
    while (!PIR1bits.SSPIF) { if (--t == 0u) return 0u; }
    PIR1bits.SSPIF = 0;
    return (uint8_t)(!PIR2bits.BCLIF && !SSPCON2bits.ACKSTAT);
}
```

```
static uint8_t i2c_read(uint8_t *data, uint8_t nack) {
    if (!i2c_cmd(0x08u) || !SSPSTATbits.BF) return 0u;
    *data = SSPBUF;
    SSPCON2bits.ACKDT = nack ? 1 : 0;
    return i2c_cmd(0x10u);
}

static uint8_t i2c_write(uint8_t addr7, const uint8_t *data, uint8_t len) {
    uint8_t ok = (uint8_t)(i2c_start() && i2c_send((uint8_t)(addr7 << 1)));
    while (ok && len-- != 0u) ok = i2c_send(*data++);
    if (!i2c_stop()) ok = 0u;
    return ok;
}

static uint8_t i2c_scan_lcd(void) {
    uint8_t address, candidate = 0u;
    i2c_devices_found = 0u;

    for (address = 0x08u; address <= 0x77u; ++address) {
        if (i2c_write(address, 0, 0u)) {
            if (i2c_devices_found < 255u) ++i2c_devices_found;

            if ((address >= LCD_ADDR_PCF8574_MIN && address <= LCD_ADDR_PCF8574_MAX) ||
                (address >= LCD_ADDR_PCF8574A_MIN && address <= LCD_ADDR_PCF8574A_MAX)) {
                if (candidate == 0u || address == 0x27u || (address == 0x3Fu && candidate != 0x27u))
                    candidate = address;
            }
        }
    }
    return candidate;
}

/* LCD I2C */
static void lcd_byte(uint8_t v, uint8_t rs) {
    uint8_t b[4], d, nib = (uint8_t)(v & 0xF0u), i;
    if (!lcd_ok) return;
    for (i = 0u; i < 4u; i = (uint8_t)(i + 2u)) {
        d = (uint8_t)(nib | rs | lcd_backlight);
        b[i] = (uint8_t)(d | LCD_EN);
        b[(uint8_t)(i + 1u)] = d;
        nib = (uint8_t)(v << 4);
    }
    if (!i2c_write(lcd_addr, b, 4u)) { lcd_ok = 0u; return; }
    if (rs == 0u && v <= 0x02u) __delay_ms(2);
    else __delay_us(40);
}

#define lcd_char(c) lcd_byte((uint8_t)(c), LCD_RS)
#define lcd_clear() lcd_byte(0x01u, 0u)
#define lcd_line(row) lcd_byte((uint8_t)(0x80u | ((row) << 6)), 0u)

static void lcd_u2(uint8_t v) {
    put2(30u, v);
    lcd_char(work_buf[30]);
    lcd_char(work_buf[31]);
}

static void lcd_hex_nibble(uint8_t n) {
    n &= 0x0Fu;
    lcd_char(n + (n < 10u ? '0' : ('A' - 10u)));
}

static void lcd_str(const char *s) {
    while (*s && lcd_ok) lcd_char(*s++);
}

static void lcd_nl(const char *s) {
    lcd_line(1u);
    lcd_str(s);
}

static void lcd_su2(const char *s, uint8_t v) {
```

```
    lcd_str(s);
    lcd_u2(v);
}

static void lcd_bl_apply(void) {
    if (lcd_addr != 0u) i2c_write(lcd_addr, &lcd_backlight, 1u);
}

static void lcd_init(uint8_t address) {
    static const uint8_t init_cmds[6] = {0x33u, 0x32u, 0x28u, 0x0Cu, 0x06u, 0x01u};
    uint8_t i;
    if (address == 0u) { lcd_ok = 0u; return; }
    lcd_addr = address;
    lcd_ok = 1u;
    lcd_backlight = LCD_BL;
    delay_ms(50u);
    for (i = 0u; i < 6u; ++i) { lcd_byte(init_cmds[i], 0u); delay_ms(5u); }
}

static void lcd_decimal10(uint16_t x10) {
    uint8_t whole = 0u;
    while (x10 >= 10u) { x10 -= 10u; ++whole; }
    lcd_u2(whole);
    lcd_char('.');
    lcd_char('0' + (uint8_t)x10);
}

/* DS1307 */
static uint8_t bcd_to_dec(uint8_t v) {
    uint8_t hi = (uint8_t)(v >> 4), lo = (uint8_t)(v & 0x0Fu);
    if (hi > 9u || lo > 9u) return 0xFFu;
    return (uint8_t)((((hi << 2) + hi) << 1) + lo);
}

static uint8_t rtc_read(void) {
    static const uint8_t rtc_mask[7] = {0x7Fu, 0x7Fu, 0x3Fu, 0x07u, 0x3Fu, 0x1Fu, 0xFFu};
    static const uint8_t rtc_max[7] = {23u, 59u, 59u, 7u, 31u, 12u, 99u};
    uint8_t i, b, h, ch = 0u;

    if (!i2c_start() || !i2c_send(0xD0u) || !i2c_send(0x00u) ||
        !i2c_restart() || !i2c_send(0xD1u)) {
        i2c_stop();
        return 1u;
    }
    for (i = 0u; i < 7u; ++i) {
        if (!i2c_read(&b, (uint8_t)(i == 6u))) { i2c_stop(); return 1u; }
        if (i == 0u) ch = (uint8_t)(b & 0x80u);
        if (i == 2u && (b & 0x40u)) {
            h = bcd_to_dec((uint8_t)(b & 0x1Fu));
            if (h < 0xFFu) {
                if (b & 0x20u) { if (h < 12u) h += 12u; } else if (h == 12u) h = 0u;
            }
        } else h = bcd_to_dec((uint8_t)(b & rtc_mask[i]));
        rtc_time[i] = h;
    }
    if (!i2c_stop()) return 1u;
    if (ch) return 2u;
    b = rtc_time[0]; rtc_time[0] = rtc_time[2]; rtc_time[2] = b;
    for (i = 0u; i < 7u; ++i)
        if (i != 3u && (rtc_time[i] > rtc_max[i] || ((i == 4u || i == 5u) && rtc_time[i] == 0u)))
            return 3u;
    if (rtc_year < 26u) return 3u;
    return 0u;
}

/* DHT11 */
static uint8_t dht_wait(uint8_t level, uint8_t timeout_us) {
    while (timeout_us-- != 0u) {
        if ((uint8_t)DHT_PIN == level) return 1u;
        __delay_us(1);
    }
    return 0u;
```

```
}

static uint8_t dht_read(void) {
    uint8_t i, j, b;
    DHT_PIN = 0;
    DHT_TRIS = 0;
    __delay_ms(20);
    DHT_TRIS = 1;
    __delay_us(30);
    if (!dht_wait(0u, 100u) || !dht_wait(1u, 120u) || !dht_wait(0u, 120u)) return 1u;
    for (i = 0u; i < 5u; ++i) {
        b = 0u;
        for (j = 0u; j < 8u; ++j) {
            if (!dht_wait(1u, 100u)) return 2u;
            __delay_us(40);
            b <<= 1;
            if (DHT_PIN) b |= 1u;
            if (!dht_wait(0u, 100u)) return 2u;
        }
        work_buf[i] = b;
    }
    if ((uint8_t)(work_buf[0] + work_buf[1] + work_buf[2] + work_buf[3]) != work_buf[4]) return 3u;
    if (work_buf[0] == 0u || work_buf[0] > 99u || work_buf[2] > 60u) return 4u;
    humidity = work_buf[0];
    temperature = work_buf[2];
    return 0u;
}

/* SPI SOFTWARE */
#define sd_cs_low() do { portb_shadow &= (uint8_t)~0x08u; PORTB = portb_shadow; } while (0)
#define sd_cs_high() do { portb_shadow |= 0x08u; PORTB = portb_shadow; } while (0)

static uint8_t spi_transfer(uint8_t data) {
    uint8_t i, rx = 0u;
    for (i = 0u; i < 8u; ++i) {
        if (data & 0x80u) portb_shadow |= 0x02u;
        else portb_shadow &= (uint8_t)~0x02u;
        PORTB = portb_shadow;
        data <<= 1;
        portb_shadow |= 0x04u;
        PORTB = portb_shadow;
        __delay_us(SD_HALF_US);
        rx <<= 1;
        if (PORTBbits.RB0) rx |= 1u;
        portb_shadow &= (uint8_t)~0x04u;
        PORTB = portb_shadow;
        __delay_us(SD_HALF_US);
    }
    return rx;
}

static void spi_skip(uint16_t n) {
    while (n-- != 0u) spi_transfer(0xFFu);
}

static void sd_deselect(void) {
    sd_cs_high();
    spi_transfer(0xFFu);
}

/* Devuelve R1 dejando CS activo para cualquier respuesta adicional. */
static uint8_t sd_command(uint8_t cmd, uint32_t arg) {
    uint16_t t = 1000u;
    uint8_t i, r = 0xFFu, crc = 0xFFu;
    sd_cs_low();
    while (t-- != 0u) {
        if (spi_transfer(0xFFu) == 0xFFu) break;
        if (t == 0u && cmd != 0u) return 0xFFu;
    }
    if (cmd == 0u) crc = 0x95u;
    else if (cmd == 8u) crc = 0x87u;
    spi_transfer((uint8_t)(0x40u | cmd));
```

```
    for (i = 4u; i != 0u; ) spi_transfer(((uint8_t *)&arg)[--i]);
    spi_transfer(crc);
    for (i = 0u; i < 10u; ++i) {
        r = spi_transfer(0xFFu);
        if (!(r & 0x80u)) break;
    }
    return r;
}

static uint8_t sd_command_end(uint8_t c, uint32_t arg) {
    uint8_t r = sd_command(c, arg);
    sd_deselect();
    return r;
}

static uint8_t sd_init(void) {
    uint8_t i, r, v2 = 0u, b2, b3;
    uint16_t t;
    sd_sdhc = 0u;
    sd_cs_high();
    spi_skip(10u);
    r = 0xFFu;
    for (i = 0u; i < 10u && r != 1u; ++i) r = sd_command_end(0u, 0UL);
    if (r != 1u) return 1u;

    r = sd_command(8u, 0x01AAUL);
    if (r == 1u) {
        spi_skip(2u);
        b2 = spi_transfer(0xFFu);
        b3 = spi_transfer(0xFFu);
        sd_deselect();
        if (b2 == 0x01u && b3 == 0xAAu) v2 = 1u;
    } else sd_deselect();

    r = 0xFFu;
    for (t = 0u; t < 1500u; ++t) {
        if (sd_command_end(55u, 0UL) > 1u) break;
        r = sd_command_end(41u, v2 ? 0x40000000UL : 0UL);
        if (r != 1u) break;
        __delay_ms(1);
    }
    if (r != 0u) {
        r = 1u;
        for (t = 0u; t < 1500u; ++t) {
            r = sd_command_end(1u, 0UL);
            if (r == 0u) break;
            __delay_ms(1);
        }
    }
    if (r != 0u) return 1u;

    if (v2) {
        r = sd_command(58u, 0UL);
        if (r == 0u) {
            if (spi_transfer(0xFFu) & 0x40u) sd_sdhc = 1u;
            spi_skip(3u);
        }
        sd_deselect();
    }
    if (!sd_sdhc) (void)sd_command_end(16u, 512UL);
    return 0u;
}

static __uint24 log_base = LOG_START_SECTOR;

static uint32_t sd_address(__uint24 sector) {
    return sd_sdhc ? (uint32_t)sector : ((uint32_t)sector << 9);
}

/* Comando de bloque; si falla reintenta con el otro modo de direccion. */
static uint8_t sd_block_cmd(uint8_t cmd, __uint24 sector) {
    if (sd_command(cmd, sd_address(sector)) != 0u) {
```

```
        sd_deselect();
        sd_sdhc ^= 1u;
        if (sd_command(cmd, sd_address(sector)) != 0u) { sd_deselect(); return 0u; }
    }
    return 1u;
}

/* Lee o verifica los primeros 32 bytes de work_buf y descarta el resto del sector. */
static uint8_t sd_read_or_verify(__uint24 sector, uint8_t verify) {
    uint16_t i;
    uint8_t token = 0xFFu, b, ok = 1u;
    if (!sd_block_cmd(17u, sector)) return 0u;
    for (i = 0u; i < 10000u; ++i) {
        token = spi_transfer(0xFFu);
        if (token != 0xFFu) break;
    }
    if (token != 0xFEu) { sd_deselect(); return 0u; }
    for (i = 0u; i < 32u; ++i) {
        b = spi_transfer(0xFFu);
        if (verify) { if (work_buf[i] != b) ok = 0u; } else work_buf[i] = b;
    }
    spi_skip(482u);
    sd_deselect();
    return ok;
}

/* Escribe 32 bytes de work_buf, completa con ceros y verifica la escritura.
 * Devuelve 1 solo si el dato leido de vuelta coincide. */
static uint8_t sd_write_verify(__uint24 sector) {
    uint16_t i;
    uint8_t r = 0xFFu;
    if (!sd_block_cmd(24u, sector)) return 0u;
    spi_skip(1u);
    spi_transfer(0xFEu);
    for (i = 0u; i < 32u; ++i) spi_transfer(work_buf[i]);
    for (i = 0u; i < 480u; ++i) spi_transfer(0x00u);
    spi_skip(2u);
    for (i = 0u; i < 10u; ++i) {
        r = spi_transfer(0xFFu);
        if (r != 0xFFu) break;
    }
    if ((r & 0x1Fu) == 0x05u) {
        for (i = 0u; i < 30000u; ++i)
            if (spi_transfer(0xFFu) == 0xFFu) { sd_deselect(); return sd_read_or_verify(sector, 1u); }
    }
    sd_deselect();
    return 0u;
}

/* REGISTRO */
static void put6(uint8_t pos, __uint24 v) {
    uint8_t hi = 0u, mid = 0u;
    uint16_t rem;
    while (v >= 10000U) { v -= 10000U; ++hi; }
    rem = (uint16_t)v;
    while (rem >= 100u) { rem -= 100u; ++mid; }
    put2(pos, hi);
    put2((uint8_t)(pos + 2u), mid);
    put2((uint8_t)(pos + 4u), (uint8_t)rem);
}

static const char idx_tag[4] = {'I', 'D', 'X', ':'};

static uint8_t load_index(void) {
    uint8_t i, d, k;
    __uint24 n = 0U, prev;
    record_count = 0U;
    log_base = LOG_START_SECTOR;
    if (!sd_read_or_verify(LOG_START_SECTOR - 1UL, 0u)) {
        log_base = 2UL;
        if (!sd_read_or_verify(1UL, 0u)) return 1u;
    }
```

```
    for (i = 0u; i < 4u; ++i) if (work_buf[i] != (uint8_t)idx_tag[i]) return 1u;
    for (i = 4u; i < 10u; ++i) {
        d = (uint8_t)(work_buf[i] - '0');
        if (d > 9u) return 1u;
        prev = n;
        for (k = 0u; k < 9u; ++k) n += prev;
        n += d;
    }
    if (n <= LOG_MAX_RECORDS) record_count = n;
    return 1u;
}

static void sd_mount(void) {
    sd_ready = 0u;
    sd_error = sd_init();
    if (sd_error != 0u) return;
    if (!load_index()) { sd_error = 3u; return; }
    sd_ready = 1u;
    sd_error = 0u;
}

/* ESTADISTICAS Y PANTALLAS */
static uint16_t mul10(uint8_t v) {
    uint16_t x = v;
    return (uint16_t)(((x << 2) + x) << 1);
}

static const uint8_t rmap[6] = {4u, 5u, 6u, 0u, 1u, 2u};
static uint8_t rtc_needs_sd = 0u;

/* Lee el DS1307; sin bateria restaura la ultima fecha de la microSD o una por defecto */
static uint8_t rtc_update(void) {
    static const uint8_t def_time[7] = {11u, 55u, 0u, 1u, 21u, 10u, 26u}; /* 11:55:00, 21/10/26 */
    uint8_t r = rtc_read(), i, d1, d0, pos = 7u;
    if (r >= 2u || (rtc_needs_sd && sd_ready)) {
        for (i = 0u; i < 7u; ++i) rtc_time[i] = def_time[i];
        rtc_needs_sd = (uint8_t)!sd_ready;
        if (sd_ready && record_count > 0U && sd_read_or_verify(log_base + record_count - 1UL, 0u)) {
            for (i = 0u; i < 6u; ++i) {
                d1 = (uint8_t)(work_buf[pos] - '0');
                d0 = (uint8_t)(work_buf[(uint8_t)(pos + 1u)] - '0');
                if (d1 > 9u || d0 > 9u) break;
                work_buf[i] = (uint8_t)((uint8_t)mul10(d1) + d0);
                pos = (uint8_t)(pos + 3u);
            }
            if (i == 6u && work_buf[2] >= 26u) {
                for (i = 0u; i < 6u; ++i) rtc_time[rmap[i]] = work_buf[i];
            }
        }
        work_buf[0] = 0x00u;
        for (i = 0u; i < 7u; ++i) {
            d0 = rtc_time[(i == 0u) ? 2u : ((i == 2u) ? 0u : i)];
            d1 = 0u;
            while (d0 >= 10u) { d0 -= 10u; d1 += 0x10u; }
            work_buf[(uint8_t)(i + 1u)] = (uint8_t)(d1 | d0);
        }
        i2c_write(0x68u, work_buf, 8u);
        r = rtc_read();
    }
    return (uint8_t)(r == 0u);
}

/* Impresion de numeros de varios digitos */
static void lcd_u4(uint16_t v) {
    uint8_t hi = 0u;
    if (v > 9999u) v = 9999u;
    while (v >= 100u) { v -= 100u; ++hi; }
    lcd_u2(hi);
    lcd_u2((uint8_t)v);
}

static void lcd_u6(__uint24 v) {
```

```
    uint8_t i;
    put6(0u, v);
    for (i = 0u; i < 6u; ++i) lcd_char(work_buf[i]);
}

static void lcd_err(uint8_t e) {
    lcd_char('E');
    lcd_char('0' + e);
}

static void lcd_dht_status(void) {
    if (dht_error == 0u) lcd_str("OK");
    else lcd_err(dht_error);
}

/* Vista 0: ACTUAL */
static void show_current(void) {
    lcd_str("T:");
    if (dht_error == 0u) lcd_u2(temperature); else lcd_str("--");
    lcd_str("C H:");
    if (dht_error == 0u) lcd_u2(humidity); else lcd_str("--");
    lcd_str("% ");
    lcd_dht_status();
    lcd_nl("N:");
    lcd_u6(record_count);
    lcd_str(" SD:");
    if (sd_ready) lcd_str("OK ");
    else { lcd_err(sd_error); lcd_char(' '); }
}

/* Vista 2: MUESTRAS */
static void show_samples(void) {
    lcd_su2("RAM:", history_count);
    lcd_str("/24 S:");
    lcd_u4(sess_saved);
    lcd_nl("SD:");
    lcd_u6(record_count);
    lcd_str(" DHT:");
    lcd_dht_status();
}

/* Vista 3: ULTIMO REGISTRO (leido de la SD) */
static void show_lastrec(void) {
    uint8_t i;
    if (!sd_ready) {
        lcd_str("SD NO LISTA");
        lcd_line(1u);
        lcd_err(sd_error);
        return;
    }
    if (record_count == 0U) {
        lcd_str("SD LISTA");
        lcd_nl("0 REGISTROS");
        return;
    }
    if (!sd_read_or_verify(log_base + record_count - 1UL, 0u)) { lcd_str("ERROR LECTURA"); return; }
    for (i = 0u; i < 30u; ++i) /* "000123,27/10/26," / "11:55:00,25,60" */ {
        if (i == 16u) lcd_line(1u);
        lcd_char(work_buf[i]);
    }
}

/* Vista 4: BOTONES (diagnostico de hardware) */
static void show_buttons(void) {
    lcd_str("BOTONES 1=PULSA");
    lcd_nl("IZQ:");
    lcd_char((btn_cur & 0x10u) ? '1' : '0');
    lcd_str(" DER:");
    lcd_char((btn_cur & 0x40u) ? '1' : '0');
    lcd_str(" M:");
    lcd_char((btn_cur & 0x20u) ? '1' : '0');
}
```

```
/* Vistas 5..10: ESTADISTICAS */
static void show_stats(uint8_t temp_mode, uint8_t kind) {
    uint8_t n = history_count, base = temp_mode ? HISTORY_SIZE : 0u;
    uint8_t i, j, v, last, step_inc, v1, v2, v3;
    uint16_t sum10, mean_x10 = 0u, std_x10 = 0u, acc = 0u, v10, d, q, step;
    char ch = temp_mode ? 'T' : 'H';

    if (!n) {
        if (kind < 2u) { lcd_char(ch); lcd_char(':'); }
        lcd_str("SIN DATOS");
        lcd_nl("MUESTREANDO");
        return;
    }

    sum10 = (uint8_t)(n >> 1);
    for (i = 0u; i < n; ++i) {
        v = history[(uint8_t)(base + i)];
        sum10 += mul10(v);
        for (j = i; j > 0u && work_buf[(uint8_t)(j - 1u)] > v; --j)
            work_buf[j] = work_buf[(uint8_t)(j - 1u)];
        work_buf[j] = v;
    }

    lcd_char(ch);
    if (kind == 0u) {
        while (sum10 >= n) { sum10 -= n; ++mean_x10; }
        if (n > 1u) {
            step = (uint8_t)(n - 1u);
            step_inc = (uint8_t)(step << 1);
            for (i = 0u; i < n; ++i) {
                v10 = mul10(work_buf[i]);
                d = (v10 >= mean_x10) ? (v10 - mean_x10) : (mean_x10 - v10);
                for (q = d; q != 0u; --q) {
                    acc += d;
                    while (acc >= step) {
                        acc -= step;
                        step += step_inc;
                        ++std_x10;
                    }
                }
            }
        }
        lcd_str(": m:"); lcd_decimal10(mean_x10);
        lcd_str(" s:"); lcd_decimal10(std_x10);
        lcd_nl("N:"); lcd_u2(n); lcd_str(" MUESTRAS");
        return;
    }

    last = (uint8_t)(n - 1u);
    if (kind == 1u) {
        v1 = work_buf[0];
        v2 = work_buf[last >> 1];
        v3 = work_buf[last];
        lcd_su2(": MN:", v1); lcd_su2(" MD:", v2);
        lcd_nl("MX:");
    } else {
        v1 = work_buf[last >> 2];
        v2 = work_buf[(uint8_t)(((last << 1) + last) >> 2)];
        v3 = (uint8_t)(v2 - v1);
        lcd_su2(" Q1:", v1); lcd_su2(" Q3:", v2);
        lcd_nl("IQR:");
    }
    lcd_u2(v3); lcd_su2(" N:", n);
}

/* Vistas 11..12: PRONOSTICO */
static void show_forecast(uint8_t temp_mode) {
    uint8_t i, cur, prev, d, max_v;
    lcd_str("PRONOSTICO ");
    lcd_char(temp_mode ? 'T' : 'H');
    lcd_line(1u);
```

```
    if (fc_count < 2u) { lcd_str("ESPERA 1 HORA"); return; }
    cur = fc_last[temp_mode];
    prev = fc_prev[temp_mode];
    max_v = temp_mode ? 50u : 99u;
    for (i = 0u; i < 3u; ++i) {
        if (cur >= prev) {
            d = (uint8_t)(cur - prev);
            cur = ((uint8_t)(max_v - cur) < d) ? max_v : (uint8_t)(cur + d);
        } else {
            d = (uint8_t)(prev - cur);
            cur = (cur < d) ? 0u : (uint8_t)(cur - d);
        }
        if (i != 0u) lcd_char(' ');
        lcd_u2(cur);
    }
    lcd_str(temp_mode ? " C" : " %");
}

static void lcd_triple(uint8_t offset, char sep) {
    uint8_t i;
    for (i = 0u; i < 3u; ++i) {
        if (i != 0u) lcd_char(sep);
        if (rtc_ok) lcd_u2(rtc_time[(uint8_t)(offset + i)]);
        else lcd_str("--");
    }
}

/* Vista 1: RELOJ */
static void show_clock(void) {
    lcd_str("HORA: ");
    lcd_triple(0u, ':');
    lcd_nl("FECHA: ");
    lcd_triple(4u, '/');
}

/* PLANIFICADOR DE TAREAS (20 ms) */
static void forecast_update(uint8_t t, uint8_t h) {
    fc_prev[0] = fc_last[0];
    fc_prev[1] = fc_last[1];
    fc_last[0] = h;
    fc_last[1] = t;
    if (fc_count < 2u) ++fc_count;
}

/* Tarea 1: Muestreo del sensor DHT11 cada 10 s + actualizacion de historico */
static void task_sensor_sample(void) {
    dht_error = dht_read();
    if (dht_error == 0u) {
        history[history_pos] = humidity;
        history[(uint8_t)(history_pos + HISTORY_SIZE)] = temperature;
        if (++history_pos >= HISTORY_SIZE) history_pos = 0u;
        if (history_count < HISTORY_SIZE) ++history_count;

        if (rtc_ok) {
            soft_hour_ticks = 0u;
            if (last_hour != rtc_hour) {
                last_hour = rtc_hour;
                forecast_update(temperature, humidity);
            }
        } else if (++soft_hour_ticks >= 360u) /* 360 * 10 s = 1 hora */ {
            soft_hour_ticks = 0u;
            forecast_update(temperature, humidity);
        }
        sd_task_state = SD_TASK_WRITE_DATA;
    } else if (!sd_ready) {
        /* Si la microSD no esta lista, reintentar montaje aunque falle el DHT11 */
        sd_task_state = SD_TASK_WRITE_DATA;
    }
    needs_redraw = 1u;
}

/* Tarea 2: Maquina de estados de registro en microSD (1 sector por tick) */
```

```
static void task_sd_step(void) {
    static const char tpl[26] = ",--/--/--,--:--:--,00,00\r\n";
    uint8_t i, pos = 7u;

    if (sd_task_state == SD_TASK_WRITE_DATA) {
        if (!sd_ready) {
            if (sd_retry == 0u) {
                sd_mount();
                sd_retry = SD_RETRY_SAMPLES;
                needs_redraw = 1u;
            } else --sd_retry;
        }
        if (!sd_ready || dht_error != 0u) { sd_task_state = SD_TASK_IDLE; return; }

        if (record_count >= LOG_MAX_RECORDS) record_count = 0U;
        put6(0u, record_count + 1U);
        for (i = 0u; i < 26u; ++i) work_buf[(uint8_t)(6u + i)] = (uint8_t)tpl[i];
        if (rtc_ok) {
            for (i = 0u; i < 6u; ++i) {
                put2(pos, rtc_time[rmap[i]]);
                pos = (uint8_t)(pos + 3u);
            }
        }
        put2(25u, temperature);
        put2(28u, humidity);

        if (!sd_write_verify(log_base + record_count)) {
            sd_ready = 0u; sd_error = 2u;
            sd_task_state = SD_TASK_IDLE;
            needs_redraw = 1u;
            return;
        }
        ++record_count;
        if (sess_saved < 9999u) ++sess_saved;
        sd_task_state = SD_TASK_WRITE_INDEX;
        return;
    }

    if (sd_task_state == SD_TASK_WRITE_INDEX) {
        for (i = 0u; i < 32u; ++i) work_buf[i] = 0u;
        for (i = 0u; i < 4u; ++i) work_buf[i] = (uint8_t)idx_tag[i];
        put6(4u, record_count);
        work_buf[10] = '\r'; work_buf[11] = '\n';
        if (!sd_write_verify(log_base - 1UL)) {
            sd_ready = 0u; sd_error = 3u;
        } else sd_error = 0u;
        sd_task_state = SD_TASK_IDLE;
        needs_redraw = 1u;
    }
}

/* MAIN */
void main(void) {
    uint8_t i, cur_btns, trig_btns;
    ANSEL = 0x00u;
    ANSELH = 0x00u;
    TRISA = 0xFFu;
    TRISB = 0xF1u;
    TRISC = TRISC_INIT_MASK;
    TRISD = 0x00u;
    PORTA = 0x00u;
    PORTB = portb_shadow;
    PORTC = 0x00u;
    PORTD = 0x00u;
    OPTION_REGbits.nRBPU = 0;
    WPUB = 0xF1u;

    /* MSSP I2C Master a 100 kHz (Fosc = 20 MHz) */
    SSPCON = 0x28u;
    SSPCON2 = 0x00u;
    SSPSTAT = 0x80u;
    SSPADD = 49u;
```

```
PIR1bits.SSPIF = 0;
PIR2bits.BCLIF = 0;

/* Timer1: Fosc/4 = 5 MHz, prescaler 1:8 */
T1CON = 0x31u;
delay_ms(200u);

lcd_init(i2c_scan_lcd());
if (lcd_ok) {
    lcd_str("I2C SCAN OK");
    lcd_nl("LCD:0x");
    lcd_hex_nibble((uint8_t)(lcd_addr >> 4));
    lcd_hex_nibble(lcd_addr);
    lcd_su2(" DEV:", i2c_devices_found);
    for (i = 0u; i < 6u; ++i) delay_ms(200u);
    lcd_clear();
    lcd_str("PIC16F887 20MHz");
    lcd_nl("INICIANDO...");
}
/* La microSD va primero: de ahi se restaura la fecha si el RTC no tiene bateria */
sd_mount();
rtc_ok = rtc_update();
if (lcd_ok) { delay_ms(200u); delay_ms(200u); needs_redraw = 1u; }

btn_prev = (uint8_t)(~PORTB & 0x70u);
btn_cur = btn_prev;

while (1) {
    /* Timer1: 65536 - 12500 = 0xCF2C -> 20 ms */
    TMR1H = 0xCFu;
    TMR1L = 0x2Cu;
    PIR1bits.TMR1IF = 0;

    PORTD ^= 0x01u;

    /* Tarea A (20 ms): botones y luz automatica */
    cur_btns = (uint8_t)(~PORTB & 0x70u);
    if (cur_btns != btn_prev && page == PAGE_BTN) needs_redraw = 1u;
    btn_cur = cur_btns;
    trig_btns = (uint8_t)(cur_btns & (uint8_t)~btn_prev);
    btn_prev = cur_btns;

    if (trig_btns != 0u) {
        bl_timer = 0u;
        if (lcd_backlight == 0u) {
            lcd_backlight = LCD_BL;
            lcd_bl_apply();
        }
        if (trig_btns & 0x10u) { if (page == 0u) page = VIEW_COUNT - 1u; else --page; needs_redraw = 1u; }
        if (trig_btns & 0x40u) { if (++page >= VIEW_COUNT) page = 0u; needs_redraw = 1u; }
        if (trig_btns & 0x20u) tick_sample = SAMPLE_PERIOD_TICKS;
    }

    /* Tarea B (200 ms): RTC y apagado de luz a los 30 s */
    if (++tick_rtc >= RTC_PERIOD_TICKS) {
        tick_rtc = 0u;
        if (lcd_backlight != 0u && ++bl_timer >= BL_TIMEOUT_TICKS) {
            lcd_backlight = 0u;
            lcd_bl_apply();
        }
        rtc_ok = rtc_update();
        if (rtc_ok && rtc_sec != last_sec) {
            last_sec = rtc_sec;
            if (page <= 1u) needs_redraw = 1u;
        }
    }

    /* Tareas C/D: sensor cada 10 s y escritura escalonada en microSD */
    if (++tick_sample >= SAMPLE_PERIOD_TICKS) {
        tick_sample = 0u;
        task_sensor_sample();
    } else if (sd_task_state != SD_TASK_IDLE) {
```

```
            task_sd_step();
        }

        /* Tarea E: refresco del LCD sin parpadeo */
        if (needs_redraw && lcd_ok) {
            needs_redraw = 0u;
            if (page != last_page || page >= 2u) {
                lcd_clear();
                last_page = page;
            } else lcd_line(0u);

            if (page == 0u) show_current();
            else if (page == 1u) show_clock();
            else if (page == 2u) show_samples();
            else if (page == 3u) show_lastrec();
            else if (page == 4u) show_buttons();
            else if (page < 11u) {
                i = (uint8_t)(page - 5u); /* 0..2 = temperatura, 3..5 = humedad */
                show_stats((uint8_t)(i < 3u), (uint8_t)((i < 3u) ? i : (i - 3u)));
            } else show_forecast((uint8_t)(page == 11u));
        } else if (!lcd_ok && ++lcd_retry_ticks >= LCD_RETRY_PERIOD) {
            lcd_retry_ticks = 0u;
            lcd_init(i2c_scan_lcd());
            if (lcd_ok) { bl_timer = 0u; last_page = 0xFFu; needs_redraw = 1u; }
        }

        while (!PIR1bits.TMR1IF) {}
    }
}
```